

第四单元：函数

- 1. 认识函数
- 2. 定义函数
 - 2.1. 函数声明
 - 2.2. 函数表达式
- 3. 函数的参数
- 4. 函数的返回值
- 5. arguments实参集合
- 6. 作用域
 - 6.1. 认识作用域
 - 6.2. 作用域链
- 7. 总结

1. 认识函数

- 函数：执行特定任务或计算返回值的代码块，可以多次调用。
- 作用：
 - 代码复用：相同的代码可以在多个地方被调用，减少重复代码。
 - 模块化：将大问题分解为小块，每块用一个函数解决。
 - 抽象：隐藏具体实现细节，提供简洁的接口。

2. 定义函数

2.1. 函数声明

- 基本语法：函数声明会被提升，在函数声明的前后调用都可以

▼ function

JavaScript |

```
1 ▼ function 函数名() {  
2     // 代码内容  
3 }  
4 // 调用函数  
5 函数名()
```

- 示例:

▼ function

JavaScript |

```
1 // 可以在函数声明之前调用函数  
2 logNum()  
3 ▼ function logNum() {  
4 ▼     for (let i = 0; i < 5; i++) {  
5         console.log(i);  
6     }  
7 }  
8 // 调用此函数会执行函数内部的for循环  
9 logNum()
```

2.2. 函数表达式

- 基本语法: 只能在函数表达式定义之后调用, 否则报错, 必须先定义后使用

▼ function

JavaScript |

```
1 ▼ var 函数名 = function () {  
2     // 代码内容  
3 }  
4 // 调用函数  
5 函数名()
```

- 示例:

```
1 // logNum() 在此处调用函数会报错
2 var logNum = function () {
3   for (let i = 0; i < 5; i++) {
4     console.log(i);
5   }
6 }
7 // 调用函数
8 logNum()
```

3. 函数的参数

- 定义函数时函数中某个或某些值不确定时可以使用函数的参数当作变量，调用时传入具体的值，多个参数用“,”分隔
- **形参：形式参数，定义函数时传入**
- **实参：实际参数，调用函数时传入**

```
1 // 打印0-某个数之间的数字
2 function logNum(params1, params2) {
3   for (let i = 0; i < params1; i++) {
4     console.log(i, params2);
5   }
6 }
7 // 调用函数时传入实参替换函数中的行参，执行代码
8 logNum(10, '我是第二个参数') // 打印 0 - 10
9 logNum(20, '我是第二个参数') // 打印 0 - 20
```

4. 函数的返回值

- 函数的返回值是函数执行后返回给调用者的数据
- 函数通过 `return` 语句返回值，如果没有 `return` 语句或者 `return` 后没有值，则返回 `undefined`。
- `return` 后的代码不会执行

```
1 function double(num) {  
2   console.log('执行函数')  
3   return 2 * num  
4   console.log('end') // 此行代码不会执行  
5 }  
6 // 调用函数时可以接收函数的返回值  
7 var result = fn(2) // result === 4
```

5. arguments实参集合

- **arguments** 是一个类数组对象，它代表了传递给函数的所有实参列表。
- **访问参数**：使用索引访问传递给函数的每个参数，如**arguments[0]**是第一个参数。
- **获取参数长度**：**arguments.length**属性提供了传递给函数的参数个数。

```
1 function fn() {  
2   console.log(arguments) // [1,2,3,4,5,6,7]  
3   console.log(arguments.length) // 获取参数的长度  
4   console.log(arguments[0]) // 获取第一个参数  
5 }  
6 var result = fn(1, 2, 3, 4, 5, 6, 7)
```

6. 作用域

6.1. 认识作用域

- 作用域是针对变量而言的，表示变量生效的范围。
- 在 JavaScript 中以函数为界，作用域分为两种
 - **全局作用域**：
 - 在代码的任何地方都能访问到的变量是**全局变量**。
 - 在代码最外层定义的变量或函数属于全局作用域。
 - **函数作用域**
 - 只在函数内部可访问的变量是**局部变量**。
 - 在函数内部定义的变量或函数只能在该函数内部被访问，属于函数作用域。

- 两个函数内部定义的同名变量属于不同的作用域，互相独立、互不影响。

• 示例：

```
▼ 作用域 JavaScript |
1  var globalVar = "这是一个全局变量";
2
3  function myFunction() {
4      var localVar = "这是一个局部变量";
5      console.log(globalVar); // 可访问
6      console.log(localVar); // 可访问
7  }
8
9  myFunction();
10 console.log(globalVar);      // 可访问
11 console.log(localVar);      // 不可访问，会报错
```

6.2. 作用域链

- 当访问一个变量时，会先在当前作用域中查找，如果没有找到，它会继续在上一层作用域中查找，直到找到该变量或者达到全局作用域。

7. 总结

1. 函数声明和函数表达式两种方式定义函数。
2. 函数的参数，形参和实参。
3. return 是函数的返回值，return 后的代码不会执行。
4. arguments 访问实参集合。
5. 在函数体内声明的变量为局部变量，不再函数内声明的变量为全局变量。
6. 局部变量只能在函数体内使用，全局变量可以在任意地方使用。